

Свиночервь Тамагочи — План развития:

архитектура, метапрогрессия и экономика

Статус документа: черновик направления развития, версия 2. **Назначение:** зафиксировать целевую архитектуру и экономику, чтобы инфраструктура выстраивалась под них уже сейчас, пока игра в зачаточном состоянии, а не переписывалась потом. **Важно:** конкретные цифры баланса — ориентировочные. Не хардкодить их по коду мини-игр, выносить в конфиги.

0. Главное изменение по сравнению с текущим состоянием

Игра — **Telegram Web App с серверной частью**. Это не «сайт с localStorage», и проектировать её так нельзя.

Два следствия, которые определяют всю архитектуру:

1. **Всё, что завязано на время, идёт от реального времени и работает при закрытой игре.** Потребности червя падают, пока приложение не запущено. Оффлайн-доход капает. Суточные лимиты и стрики считаются по календарю, а не по сессиям.
2. **Источник правды — сервер и его база данных.** Клиент — только отображение и ввод. `localStorage` перестаёт быть хранилищем состояния и становится кэшем для мгновенной отрисовки до прихода ответа сервера.

Это надо заложить **сейчас**. Позже перевод на серверную модель будет означать переписывание каждого модуля.

1. Что не так с текущей структурой

```
// main.js — так больше нельзя
setInterval(() => this.decaySins(), 500);

decaySins() {
  this.sins[key].value -= this.sins[key].decay * 0.5;
}
```

Проблемы:

- Шкала падает **только пока открыто приложение**. Закрыл — заморозил. Это противоречит самой идее тамагочи.
- Состояние живёт в оперативной памяти и умирает при закрытии.
- Мини-игры пишут в `GameManager.sins.*.value` напрямую — награды размазаны по семи файлам.
- Нет валют, нет сохранения, нет причин зайти завтра.
- Сохраняется только модель персонажа (`worm-model.js` → `localStorage`).

Всё это надо заменить до того, как поверх нарастёт метапрогрессия.

2. Принцип «ленивого времени»: никаких тикающих таймеров состояния

Ключевой архитектурный приём. Состояние **не обновляется по таймеру**. Вместо этого хранится последнее известное значение и метка времени, а актуальное значение вычисляется формулой в момент обращения.

```
value_now = clamp(0, max, value_at - decay_rate * (now - updated_at))
```

Что это даёт:

- Шкалы падают одинаково независимо от того, открыта игра или нет
- Никаких фоновых процессов на сервере, крутящих миллионы записей раз в секунду
- Один и тот же расчёт на клиенте (для плавной анимации) и на сервере (для правды)

Правила:

- `now` — **всегда серверное время**. Клиентские часы подделываются переводом системного времени. Клиент при старте получает от сервера `timestamp` и хранит поправку `serverOffset`, применяя её ко всем локальным расчётам
- То же самое касается оффлайн-дохода, кулдаунов, стриков, суточных лимитов
- Таймеры на клиенте остаются **только для анимации**: `requestAnimationFrame`

плавно двигает полосу между значениями, но не является источником данных

Оффлайн-доход и оффлайн-падение считаются **при заходе**, разово, за прошедший интервал. Оффлайн-доход имеет потолок (~8 часов), падение шкал — нет (но есть пол в 0).

3. Клиент-серверная архитектура

3.1 Разделение

Слой	Где	Ответственность
Клиент	GitHub Pages (статика)	рендер, ввод, анимация, локальный кэш, предсказание значений
API	отдельный хост	правда о состоянии, начисления, лимиты, валидация
БД	PostgreSQL	хранение
Бот	Telegram Bot API	вход в приложение, push-уведомления

GitHub Pages не умеет бэкенд — API живёт на отдельном домене (Railway / Fly.io / VPS / любой хостинг Node). Понадобится CORS.

3.2 Аутентификация

Telegram Web App передаёт в приложение `initData`. Сервер обязан проверять его подпись HMAC-SHA256 на секрете бота, а не доверять переданному `user.id`. Без этой проверки любой может выдать себя за любого игрока.

- Клиент отправляет `initData` один раз, получает сессионный токен (JWT, короткий TTL)
- Все дальнейшие запросы идут с токеном
- `telegram_user_id` — первичный ключ игрока

3.3 Модель доверия

Мини-игры считаются на клиенте, значит результат подделываем. На текущем этапе это допустимо, но архитектура должна быть готова к ужесточению:

- Сервер **никогда** не принимает «начисли мне N золота». Клиент присылает

результат (`sin` , `mode` , `outcome` , метаданные), сервер сам решает, что начислить, по своему конфигу наград

- Все конфиги наград и цен живут **на сервере**. Клиент получает их копию только для отображения
- Каждый запрос на начисление несёт `client_request_id` (UUID). Повтор с тем же `id` не начисляет второй раз — защита от двойных тапов, потерянных ответов и ретраев
- Rate limit по частоте: бой не может завершиться быстрее физического минимума
- Все начисления пишутся в **лог транзакций** (`ledger`). Без него невозможно ни разобрать баг, ни откатить накрутку

3.4 Push-уведомления

Отдельное преимущество Telegram: бот может написать игроку. Это главный инструмент возврата.

- «Свиночервь голодает» при падении шкалы ниже порога
- «Оффлайн-кошелёк заполнен»
- «Стрик сгорит через 3 часа»

Реализуется фоновой задачей, которая раз в N минут выбирает игроков, у кого расчётное значение пересекло порог и кому ещё не отправлялось уведомление в этом цикле. Обязательны настройка отключения и жёсткий лимит частоты, иначе игрок заблокирует бота.

4. Схема базы данных (черновик)

Все временные поля — `timestampz` , всё в UTC. Все денежные значения — целые числа, никаких `float`.

```
players
  id                bigserial PK
  telegram_user_id  bigint UNIQUE NOT NULL
  username          text
  created_at        timestampz
  last_seen_at      timestampz      -- база для оффлайн-расчётов
  timezone_offset   int             -- для «суток» в стриках и ежедневках
  schema_version    int
```

```

currencies
    player_id      bigint FK
    currency       text
    amount         bigint
    PRIMARY KEY (player_id, currency)
-- по строке на валюту, а не колонки

sins
    player_id      bigint FK
    sin            text
    value          numeric
    max_value      numeric
    decay_rate     numeric
    updated_at     timestampz
    PRIMARY KEY (player_id, sin)
-- 'wrath','greed',...
-- значение НА МОМЕНТ updated_at
-- растёт от мета-апгрейдов
-- единиц в секунду
-- ключевое поле для ленивого расчёта

sin_upgrades
    player_id, sin, upgrade_key, level
    PRIMARY KEY (player_id, sin, upgrade_key)
-- локальная прогрессия каждого греха

worm
    player_id      bigint PK FK
    model          jsonb
    stage          int
    cosmetics      jsonb
-- модель персонажа
-- то, что сейчас в worm-model.js
-- стадия эволюции
-- надетое по слотам

scars
    id            bigserial PK
    player_id      bigint FK
    zone          text
    t             numeric
    seed          bigint
    created_at     timestampz
-- шрамы гнева
-- 'head' | 'body' | 'tail'
-- позиция вдоль скелета, 0..1
-- детерминированная генерация формы

inventory
    player_id, item_key, qty

unlocks
    player_id, unlock_key, acquired_at
-- купленная косметика, открытые режимы

runs
    player_id      bigint FK
    sin            text
    state          jsonb
    difficulty     int
    started_at, updated_at
    PRIMARY KEY (player_id, sin)
-- незавершённые забеги рогалика
-- карта, узел, HP, зубы, предметы, дебаф

```

```

daily_counters                                -- суточные лимиты и стрики
  player_id, day_key, counter_key, value
  PRIMARY KEY (player_id, day_key, counter_key)

pvp_snapshots                                -- «слепки» для асинхронного PvE
  player_id PK, snapshot jsonb, updated_at

ledger                                         -- журнал всех начислений и трат
  id bigserial PK
  player_id, currency, delta, reason, client_request_id UNIQUE, created_at

```

Замечания по схеме:

- `currencies` — строками, а не колонками. Добавление валюты нового греха не должно требовать миграции таблицы
- `sins.updated_at` — сердце ленивого расчёта. Обновляется при каждой записи значения
- `daily_counters.day_key` вычисляется с учётом `timezone_offset` игрока, иначе «сутки» будут неверными для половины мира
- `ledger.client_request_id UNIQUE` — та самая идемпотентность
- Обязательны **миграции с номером версии**. Схема будет меняться десятки раз

5. API (черновик)

```

POST /auth/telegram      -> { token, serverTime, config }
GET  /state              -> полное состояние + serverTime
POST /minigame/result    -> { sin, mode, outcome, meta, client_request_id }
POST /shop/purchase      -> { shop, item_key, client_request_id }
POST /run/start          -> { sin, difficulty, client_request_id }
POST /run/node           -> { node_id, choice, client_request_id }
POST /run/abandon
POST /scars/exchange     -> { count, client_request_id }
POST /daily/claim

```

Принципы:

- `/state` — единственный способ узнать правду. Клиент запрашивает при старте, при возврате из фона (`visibilitychange`) и после каждого начисления

- Ответ всегда содержит `serverTime` — клиент пересчитывает `serverOffset`
 - Между запросами клиент **предсказывает** значения теми же формулами локально, чтобы полоски двигались плавно. При расхождении с ответом сервера — плавно подтягивается к серверному значению, а не прыгает
-

6. Целевая структура: три уровня прогрессии

Уровень 1 — Уход (существует)

Шкалы семи грехов падают в реальном времени, мини-игры их заполняют. Базовый цикл тамагочи.

Уровень 2 — Локальная прогрессия греха (новое)

У каждого греха своя внутренняя валюта, свой магазин и свои апгрейды, работающие **только внутри этого греха**. Каждая мини-игра становится маленькой самостоятельной игрой.

Уровень 3 — Мета-прогрессия свиночервя (новое)

Общие валюты, косметика, апгрейды персонажа, эволюция. Всё, что видно на главном экране.

Уровни не конкурируют: игрок, который любит только бои, развивает гнев и всё равно приносит золото в общую мету.

7. Система валют

7.1 Глобальные

Валюта	Назначение	Источники	Стоки
Золото	основная мягкая	победы, рогалик, ежедневки, оффлайн-доход	косметика, мета- апгрейды, расходники
Кристаллы	премиальная, редкая	ограниченные достижения, PvP- винстрик (1 раз в сутки), высокие сложности, будущий донат	эксклюзивная косметика, ускорения,

			слоты
--	--	--	-------

- Кристалл **никогда** не должен добываться простым повторением. Только через условие, невыполнимое второй раз за день
- Золото — единственное, что можно фармить, и именно на нём стоит вся косметика

7.2 Локальные валюты грехов

У каждого греха своя валюта, добываемая только в его мини-игре и тратимая только в его магазине.

- Гнев → **жетон гнева**, дробится на **осколки** (3 осколка = 1 жетон)
- Остальные шесть проектируются позже по тому же шаблону

Локальная валюта **не конвертируется** ни в золото, ни в валюту другого греха. Это замыкает каждый грех на себя и заставляет играть во все семь.

7.3 Внутрорежимные валюты

Существуют только внутри одной сессии и сгорают на выходе. Пример — **зубы** в рогалике гнева. Никогда не выносятся наружу и не конвертируются: как только появится обмен на золото, смысл «тратить прямо сейчас» умрёт.

8. Мета-прогрессия: на что тратится золото и кристаллы

8.1 Косметика (бесконечный сток)

Основная долгосрочная цель. Расширяется бесконечно, не влияет на баланс.

- **Фоны** комнаты (подвал, хлев, казино, преисподняя, космос...)
- **Одежда и аксессуары**: слоты «голова», «шея», «тело», «хвост»
- **Скины кожи**: цвет, узор, полупрозрачная, гнилая, золотая
- **Питомцы-паразиты** рядом с червём
- **Оформление HUD**
- **Боевые эффекты** — за локальную валюту греха

Технически: точки крепления косметики завязываются на **скелет** в `worm-model.js`, а не на абсолютные координаты. Иначе всё поедет при взрослении и эволюции.

8.2 Апгрейды персонажа

- скорость и потолок оффлайн-накопления
- дополнительные слоты косметики
- множитель золота
- снижение `decay_rate` шкал
- расширение шкалы выше 100% — **не как «больше запас», а как порог сверхнасыщения**: заполнил выше 100% → червь входит в особое состояние и все мини-игры сутки дают бонус к золоту. Иначе апгрейд просто сокращает число сессий, что для тамагочи вредно

8.3 Долгосрочная цель

Эволюция по стадиям (личинка → свиночервь → жирный прелат → нечто). Каждая требует прокачки всех семи грехов до порога и меняет модель персонажа. Стыкуется с запланированным анатомическим апгрейдом рендерера.

8.4 Тело как летопись

Ключевой принцип: **каждый грех оставляет на модели собственный след.**

- Гнев → шрамы
- Чревоугодие → жир, растянутое брюхо
- Лень → мох, наросты, паразиты
- Гордыня → золотые наконечники
- Зависть → бледность, зеленоватая кожа

Через месяцы игры два разных червя выглядят по-разному. Следы **косметические**, боевых бонусов не дают.

9. Удержание

- **Ежедневные задания**: 3 штуки, ротация. Сутки считаются по `timezone_offset` игрока
- **Оффлайн-доход** с потолком (~8 часов), начисляется разово при заходе
- **Стрик за вход** подряд

- **Push от бота** при падении шкал и заполнении кошелька
- **Недельный ивент** с временной валютой (позже)
- **Кросс-грех лут**: предметы из одной мини-игры используются в другой (семена из босса гнева — для лени). Правило: кросс-предмет — **ускоритель, а не единственный ключ**. Допускается 1–2 намеренно эксклюзивных предмета как приманка

10. ПРИМЕР ПОЛНОЙ ПРОРАБОТКИ: ГНЕВ

Референсный образец. Остальные шесть проектируются по этому шаблону позже.

10.1 Валюты

- **Осколок жетона гнева** — базовая единица
- **Жетон гнева** = 3 осколка
- **Зубы** — внутри рогалика, сгорают
- **Золото** — общее, выносятся

10.2 Режимы

1. **Бой с ботом (PvE)** — быстрый бой 1 на 1 против «слепка» другого игрока (снаряжение, шрамы, ник, статьи), управляемого ботом. Лобби всегда полное, реалтайм не нужен
2. **Бой с игроком (PvP)** — реалтайм, только по коду комнаты. Матчмейкинг не строится. Развлечение + единственный регулярный источник кристалла
3. **Рогалик** — платный вход, длинная сессия, основной источник жетонов и золота

10.3 Награды за бой 1 на 1

Исход	Шкала гнева	Золото	Осколки	Прочее
Победа	100%	да	+1	—
Ничья	+50%	нет	нет	—
Поражение	+20%	нет	+1 за каждые 3 поражения	шанс 30% на шрам

Расчёт: победа даёт 1 осколок, поражение ≈ 0.33 + шрамы. Победа примерно втрое выгоднее — сливать невыгодно, но поражение всё равно даёт прогресс.

Позже можно продавать за жетоны апгрейд «поражение тоже даёт золото» (меньше, чем победа).

Ограничение фарма: запрета на игру при полной шкале НЕТ — мини-игры самоценны. Ограничивается только внешняя валюта, убывающей доходностью золота:

- победы 1–5 за сутки: 100% золота
- 6–15: 50%
- дальше: 10%

Счётчик живёт в `daily_counters`. Шкала, осколки, шрамы каплют всегда в полном объёме.

10.4 Кристалл за PvP-винстрик

- 3 победы подряд в PvP → 1 кристалл
- **1 раз в сутки**, дальше стрик даёт золото
- все 3 оппонента **уникальны за сутки** (в лобби рядом с уже побеждённым сегодня — отметка)

Это сознательно оставляет возможность договорняка на двух аккаунтах, но потолок в 1 кристалл в день делает его безобидным — и заставляет заходить ежедневно и играть минимум 6 боёв.

10.5 Шрамы

Механика:

- Поражение в любом бою даёт шрам с шансом **30%**
- Шрам появляется в зоне **последнего полученного удара**: голова → голова; тело → сегменты от головы до живота; хвост → сегменты после живота
- Копятся постоянно, остаются навсегда
- **15 шрамов** обмениваются на **1 жетон гнева**, пачками по 15, остаток сохраняется
- Шрамы **не дают боевых бонусов**. Это косметика

Смысл обмена: не источник дохода, а кнопка «мне надоел этот вид». Порог 15

выбран, чтобы шрамы копились неделями и червь реально зарастал, а обмен был редким осознанным событием. Экран обмена показывает превью «до / после».

Технически:

- Хранение: { `zone`, `t`, `seed` } в таблице `scars` — привязка к сегменту скелета в нормализованных координатах, не в пикселях. При взрослении и эволюции шрамы едут вместе с телом
- Форма генерируется детерминированно из `seed` (длина, ширина, рваность контура, изгиб) — тем же подходом, что генерация растений в лени
- Цвет — **смещение в HSL от текущего тона кожи** (примерно `L -12%`, `S +8%`, сдвиг `H` в розовый), не абсолютное значение. Тогда шрамы подстраиваются под любой купленный скин
- Тон кожи берётся из одной точки в `worm-model.js`
- Минимальная дистанция между шрамами (`|\Delta t| > 0.04`), иначе слипнутся в пятно
- Визуальный кап ~8 на зону, сверху — только счётчик
- Слой рендера: поверх `skin`, под `surface detail`
- Решение о выпадении шрама принимает **сервер**, не клиент

10.6 Магазин гнева (за жетоны)

Живёт **внутри** экрана мини-игры, отдельной вкладкой. Каждый грех — своя экосистема.

Ассортимент (по образцу `combats.ru`):

- **Снаряжение по слотам:** оружие, шлем, перчатки, броня, щит — отдельные предметы со своими цифрами, а не абстрактные апгрейды
- **Уровни предметов** и апгрейды существующих
- **НР и защита** уровнями
- **Косметические эффекты** удара и блока — бесконечный сток
- **Пересборка:** сброс апгрейдов с возвратом ~70%
- **Расходники** для рогалика

У снаряжения будет потолок, и это нормально — но потолок должен быть **объявлен заранее**: игрок видит «оружие 5/5». Когда упирается в потолок, нужна следующая

цель: косметические эффекты и высокие сложности рогалика.

10.7 Боевая система

Референс — combats.ru: выбор зоны удара и зон блока чекбоксами, снаряжение по слотам.

Критично: **зон должно быть больше, чем доступных блоков** (бьёшь в 1 из 5, блокируешь 2 из 5). Иначе чтение противника вырождается в угадку 50/50 и глубина исчезает.

Не брать из референса: тайминги на реальные секунды и бесконечную лестницу уровней. Мобильная сессия — 1–2 минуты.

10.8 Рогалик

Вход: 1 жетон гнева.

Карта первого уровня сложности:

бой → бой

→ РАЗВИЛКА (магазин / полный хил / случайное событие)

→ МИНИ-БОСС ⇒ сразу: 1 осколок + всё накопленное золото

→ бой → бой

→ РАЗВИЛКА

→ БОСС ⇒ 1 жетон + всё накопленное золото + лут

Награды по ходу:

- за каждый выигранный рядовой бой: зубы, золото, небольшое усиление и **отхил ~20% максимального НР** — всё вместе, без выбора между ними
- золото копится, но **выдаётся только при победе над мини-боссом и боссом**. Слился до мини-босса — потерял накопленное
- осколок за мини-босса выдаётся **немедленно**

Почему хил на развилке бесплатный и полный: автоматический +20% после каждого боя означает, что аккуратный игрок приходит к развилке почти целым, и узел хила для него мёртв — он идёт в магазин или событие. Побитый вынужден лечиться и пропускает усиление. Решение переехало в сам бой, где ему и место.

Развилка:

1. **Магазин** — за зубы: бонусы к урону, защите, НР, а также золото на вынос
2. **Полный хил** — НР до максимума, больше ничего

3. **Случайное событие** — заготовленная ситуация с 2–3 вариантами выбора, каждый со своими последствиями. Не лотерея: игрок выбирает, а не дёргает рычаг.
Пример: «Гниющая туша. Съесть (+40% HP, –20% максимального HP) или пройти мимо (+15 зубов)?». Самое дешёвое место для юмора и характера свиночервя

Магазин и событие должны содержать **редкие предметы, которые больше нигде в забеге не выпадают** — иначе битый игрок всегда идёт лечиться и развилка схлопывается.

Уровни сложности: пройденная карта открывает следующую — длиннее, с двумя мини-боссами и другим боссом. Награда растёт с числом боёв. Высокие сложности могут давать кристалл.

Лут с босса: снаряжение для гнева и кросс-грех предметы (семена для лени и т.п.).

Наращение: бои после мини-босса заметно злее (минимум +50% HP и урона), иначе вторая половина ощущается как первая.

Разнообразие: нужен пул из 20–30 усилений, из которых на узле предлагается 3 на выбор. Без этого третий забег скучен. Отдельная проектная работа, сделать до кода.

Негативные эффекты обязаны быть видимыми: иконка сверху с понятным текстом.

Сохранение забега — критично. Мобильная сессия обязательно прервётся: звонок, свёрнутый Telegram, разряженный телефон. Потерять забег на 6 боёв вместе с жетоном — худший возможный опыт в игре.

- Состояние забега (карта, узел, HP, зубы, предметы, дебаффы, накопленное золото) пишется в таблицу `runs` **после каждого узла**, серверным запросом
- Вход в рогалик при незавершённом забеге показывает «Продолжить / Бросить», а не начинает новый
- Забег **не имеет таймера жизни** — можно вернуться через неделю

11. Модули клиента

```
src/core/
net.js      — HTTP-слой, токен, ретраи, client_request_id
time.js     — серверное время, serverOffset, все формулы времени
state.js    — единый стор состояния, кэш в localStorage, синк с /state
economy.js  — Economy.get/has, отображение валют (НЕ начисляет сам)
events.js   — шина событий
worm-model.js — существующая модель (ездит под state.js)
```

Развязка мини-игр и экономики

Мини-игры **не знают про экономику и про сервер**. Они бросают событие:

```
GameEvents.emit('minigame:result', {
  sin: 'wrath',
  mode: 'pve_duel',
  outcome: 'win' | 'lose' | 'draw',
  meta: { damageDealtPct, lastHitZone, turns }
})
```

Ядро ловит событие, отправляет на `/minigame/result`, получает от сервера новое состояние и обновляет UI. Все правила начисления — **на сервере, в одном конфиге**, а не разбросаны по семи модулям.

Заполнение шкал тоже переезжает туда: прямые записи вида

```
GameManager.sins.wrath.value = 100
```

 из `wrath.js` убираются полностью.

12. Порядок реализации

Этап 0 — фундамент (делать первым, до любой геймплейной фичи)

1. Бэкенд: скелет API, PostgreSQL, миграции, валидация Telegram `initData`, JWT
2. Схема БД по разделу 4
3. `time.js` — серверное время и ленивый расчёт шкал; удаление `setInterval` с `decaySins`
4. `state.js` + `net.js` — синк с `/state`, кэш, предсказание значений между запросами
5. Шина событий, подключение существующих мини-игр, вынос заполнения шкал в серверный конфиг

Этап 1 — экономика 6. Золото и кристаллы, серверные начисления, `ledger`, идемпотентность, суточные лимиты 7. Отображение валют в HUD

Этап 2 — гнев как референс 8. Шрамы: хранение, генерация, рендер в слое кожи, экран обмена 9. Локальные валюты + магазин гнева со снаряжением 10. Рогалик: карта, узлы, зубы, события, сохранение забега в `runs`

Этап 3 — мета 11. Косметика, слоты, мета-апгрейды 12. Ежедневки, оффлайн-доход, стрики, push от бота

Этап 4 — социальное 13. PvE-слепки 14. Реалтайм PvP по коду комнаты

Этап 5 15. Проработка остальных шести грехов по шаблону гнева

13. Не решено

- **Выбор стека бэкенда и хостинга.** Рекомендация: Node.js + Fastify + PostgreSQL, деплой на Railway или Fly.io — дешево, быстро поднимается, есть managed Postgres
- **Ближайшая цель первой недели.** Нужна одна заметная цель, достижимая за 2–3 дня (первое оружие, первый скин, первый пройденный рогалик). Без неё вся экономика будет ощущаться как бесконечное капание
- **Что происходит при обнулении шкалы.** Сейчас последствий нет. Нужны: визуальное состояние червя, возможно штраф, возможно платное «лечение». Без этого падающие шкалы ничего не значат
- Пул из 20–30 усилений для рогалика
- Конкретные цифры баланса
- Локальные валюты и следы на теле для остальных шести грехов
- Нарративная подача валют (кристаллы получают тематическое название)
- Политика push-уведомлений: пороги, частота, отключение